

Old | A

1

P-1025

11.24.4.8. Adding/Updating/Removing a Group

When adding a group, a client must create the group information entry before adding membership associations between the Node and the Group.

Adding a Group

Add Group Key Set Entry (if new) using the **AddKeySet Command**, followed by Group Information Entry using the **AddGroup Command**.

Updating a Group

Update the Group Key Set Entry (if changing) using the **UpdateKeySet Command**. Update the Group Information Entry using the **UpdateGroup Command**. KeySet and Group updates performed using these commands will cause the Datastore to attempt to update all Node members of the group with these changes. Any Node changes that do not complete (for example, if the Node is unreachable) will result in **Pending** status on the data in the Node Information Entry.

Removing a Group

Remove the Group Information Entry using the **UpdateGroup Command**. This command will fail if the Group is in use (and not with **PendingDelete** Status) by any Nodes.

Remove the Group Key Set Entry using the **RemoveKeySet Command**. This command will fail if this KeySet is in use by any Groups.

Following the pattern of red boxes, the purple box should have “**RemoveGroup Command**”, especially since this command actually exists.

However, we still explore the scopes of removing the **Group Information Entry** using the **UpdateGroup** command.

Now, let’s check the **UpdateGroup** command:

P-1041

1. If entry is not found, return NOT_FOUND.
2. Update the [DatastoreGroupInformationEntry](#) for the Group with the given GroupID to match the non-NULL fields passed in.
3. If any fields are changed as a result of this command:
 - a. Iterate through each Node Information Entry:
 - i. If the GroupKeySetID changed:
 - I. Add a [Section 11.24.5.3, "Datastore Node Key Entry Type"](#) with the new GroupKeySetID, and Status set to **Pending**.
 - II. Add this KeySet to the Node.
 1. If successful, Set the Status to **Committed** for this entry in the NodeKeySetList.
 2. If not successful, the pending change SHALL be applied in a subsequent [Node Refresh](#).
 - A. If the NodeKeySetList list contains an entry with the previous GroupKeySetID:
 - III. Set the Status set to **DeletePending**.
 - IV. Remove this KeySet from the Node.
 1. If successful, Remove this entry from the NodeKeySetList.
 2. If not successful, the pending change SHALL be applied in a subsequent [Node Refresh](#).
 - ii. If the GroupCAT, GroupCATVersion or GroupPermission changed:
 - A. If the ACLList contains an entry for this Group, update the ACL List Entry in the Datastore with the new values and Status **Pending**, update the ACL attribute on the given Node with the new values. If the update succeeds, set the Status to **Committed** on the ACLList Entry in the Datastore.
 - iii. If the FriendlyName changed:
 - A. Iterate through each Endpoint Information Entry:
 - I. If the GroupIDList contains an entry with the given GroupID:
 1. Update the GroupIDList Entry in the Datastore with the new values and Status **Pending**

2. Update the Groups on the given Node with the new values.
 1. If the update succeeds, set the Status to **Committed** on the GroupIDList Entry in the Datastore.
 2. If not successful, the pending change SHALL be applied in a subsequent [Node Refresh](#).

So, **UpdateGroup** command actually notes the changes of these fields (**marked inside red boxes in previous page's images**):

P-1040

11.24.7.5. UpdateGroup Command

This command SHALL be used to update a group in the [Joint Fabric](#) Datastore Cluster.

The data for this command is as follows:

ID	Name	Type	Constraint	Quality	Default	Conformance
0	GroupID	group-id	all			M
1	Friendly-Name	string	max 32	X		M
2	GroupKey-SetID	uint16	1 to 65535	X		M
3	GroupCAT	uint16	1 to 65535	X		M
4	Group-CATVersion	uint16	1 to 65535	X		M

ID	Name	Type	Constraint	Quality	Default	Conformance
5	GroupPermission	AccessControlEntryPrivilegeEnum	X			M

Now, let's check the **RemoveGroup** command:

P-1042

11.24.7.6. RemoveGroup Command

This command SHALL be used to remove a group from the [Joint Fabric](#) Datastore Cluster.

The data for this command is as follows:

ID	Name	Type	Constraint	Quality	Default	Conformance
0	GroupID	group-id	all			M

GroupID represents the unique identifier for the group to be removed from the [Joint Fabric](#) Datastore Cluster.

Upon receipt of this command, the Datastore SHALL:

1. If entry is not found, return NOT_FOUND.
2. Ensure there are no Nodes in this group. To do this:
 - a. Iterate through each Node Information Entry:
 - i. If the GroupIDList contains an entry with the given GroupID, and the entry does NOT have Status **DeletePending**, then return CONSTRAINT_ERROR.
3. Remove the [DatastoreGroupInformationEntry](#) for the Group with the given GroupID from the GroupList attribute.

It is written on P-1025 (in the very first picture of this contradiction) that: "This command will fail if the **Group** is in use (and not with **PendingDelete** Status) by any **Nodes**"

The **RemoveGroup** command takes care of that sentence via **Point 2** & takes care of removing the group via **Point 3**. This indicates that if we want to remove a group, we should use the **RemoveGroup** command.

NOTE:

The screenshots provided so far are from Matter spec v1.4. However, v1.5 has been recently released, and we confirm **the claimed mistake of writing "remove a group via UpdateGroup command" still persists in the spec v1.5.**

Code:

```
386 CHIP_ERROR
387 JointFabricDatastore::RemoveGroup(const Clusters::JointFabricDatastore::Commands::RemoveGroup::DecodableType & commandData)
388 {
389     size_t index = 0;
390     // Check if the group ID exists in the datastore
391     VerifyOrReturnError(IsGroupIDInDatastore(commandData.groupID, index) == CHIP_NO_ERROR,
392                         CHIP_ERROR_IM_STATUS_CODE_CONSTRAINT_ERROR);
393
394     // Remove the group entry from the datastore
395     auto it = mGroupInformationEntries.begin();
396     std::advance(it, index);
397
398     if (it->groupCAT == kAdminCATIdentifier || it->groupCAT == kAnchorCATIdentifier)
399     {
400         // If the group is an AdminCAT or AnchorCAT, we cannot remove it
401         return CHIP_ERROR_IM_STATUS_CODE_CONSTRAINT_ERROR;
402     }
403
404     mGroupInformationEntries.erase(it);
405
406     return CHIP_NO_ERROR;
407 }
```

Code of the **RemoveGroup** command suggests a **spec-code inconsistency**. Neither of the two checks mentioned in **Point 2** of **P-1042** is covered in code (if the Group is in use + **Status** is not **DeletePending**)

Rather, there is a check of AdminCAT & AnchorCAT which is not stated in v1.4. We found this check in **spec v1.5**.

RemoveGroup command section in spec v1.5:

11.24.7.6. RemoveGroup Command

This command SHALL be used to remove a group from the **Joint Fabric** Datastore Cluster of the accessing fabric.

The data for this command is as follows:

ID	Name	Type	Constraint	Quality	Fallback	Conformance
0	GroupID	group-id	all			M

GroupID represents the unique identifier for the group to be removed from the **Joint Fabric** Datastore Cluster.

Attempts to remove a group with GroupCAT value set to **Administrator CAT** or **Anchor CAT** SHALL fail with **CONSTRAINT_ERROR**.

Upon receipt of this command, the Datastore SHALL:

1. If entry is not found, return **NOT_FOUND**.
2. Ensure there are no Nodes in this group. To do this:
 - a. Iterate through each Node Information Entry:
 - i. If the GroupIDList contains an entry with the given GroupID, and the entry does NOT have Status **DeletePending**, then return **CONSTRAINT_ERROR**.
3. Remove the **DatastoreGroupInformationEntryStruct** for the Group with the given GroupID from the GroupList attribute.

Code Reference:

```
./src/app/server/JointFabricDatastore.cpp::UpdateGroup()  
./src/app/server/JointFabricDatastore.cpp::RemoveGroup()
```

Testing Progress

- We created a **Joint Fabric Datastore** by following the steps outlined here:
https://github.com/project-chip/connectedhomeip/blob/master/docs/guides/joint_fabric_guide.md
 - ./jfa-app --capabilities 0x4 --passcode 11022033 --discriminator 3840
--secured-device-port 5533 --rpc-server-port 33033 --KVS jfa_a_kvs
 - ./jfc-app --rpc-server-port 33033 --storage-directory jfc_a_storage_directory
--commissioner-vendor-id 0xFFF1
 - [from jfc-app] pairing onnetwork-long 1 11022033 3840 --anchor true
- After the datastore is created, we set up a lighting app and paired it with the joint-fabric controller
 - ./chip-lighting-app --capabilities 0x4 --passcode 11022044 --KVS light_a_kvs\
 - [from jfc-app] pairing onnetwork 2 11022044
- We used the relevant **Joint Fabric** cluster command to create a group and add the lighting app node to that group
 - ./jfc-app jointfabricdatastore add-group 10 "SecurityGroup" 1 0xABCD 1 3 1 0
 - Current issue: Executing this command results in an “**unsupported cluster**” error

Remaining ToDo

- Verify which command is actually being used. There is a **RemoveGroup** command in addition to the **UpdateGroup** method, and neither appears to include all the required checks
- We intend to demonstrate that, when deleting a group, the **RemoveGroup** command was meant to be used, but instead the documentation or guidance suggests using the **UpdateGroup** command
- Regardless of which method is used as per current codebase, our goal is to prove that a group can still be removed even if it is currently being used by a node

2

Possible Vulnerability:

Creating an Exchange based on an Incoming Message

The steps to create a new Exchange based on an incoming message are as follows:

1. A new Exchange and [Exchange Context](#) SHALL be created with the following settings:
 - a. The Exchange ID SHALL be set to the Exchange ID of the message.
 - b. The Exchange Role SHALL be set to the inverse of the incoming message [I Flag](#), for example set the Exchange Role to Responder if the message is from an Initiator.

- c. The Session Context SHALL be set to the Session on which the message was received.

A node SHOULD limit itself to a maximum of 5 concurrent exchanges over a unicast session. This is to prevent a node from exhausting the message counter window of the peer node.

If proper cryptographic measures are not taken (encryption, integrity check), then this “role setting based on incoming message” may cause security issue

Analysis (on how exploitation MAY be possible):

===== Part 1 =====

4.10.5. Exchange Message Processing

After completion of [Section 4.7.2, "Message Reception"](#), if the message [matches an existing Exchange](#), it is dispatched to the appropriate protocol handler in the next higher layer. Messages for an existing Exchange are dispatched to the handler for that Exchange. Otherwise, the [unsolicited message](#) that created the Exchange is dispatched to the unsolicited message handler.

4.10.5.1. Exchange Message Matching

Upon receipt of a message, the Exchange Layer attempts to match the message to an existing [Exchange](#). A given message is part of an Exchange if it satisfies all the following criteria:

1. The message was received over the session associated with the Exchange.
2. The Exchange ID of the message matches the Exchange ID of the Exchange,
3. The message has the [I Flag](#) set and the Exchange Role of the Exchange is Responder,
OR the message does not have the [I Flag](#) set and the Exchange Role of the Exchange is Initiator.

If the message does not match an existing Exchange, the message is considered an [unsolicited message](#).

4.10.5.2. Unsolicited Message Processing

An unsolicited message is processed as follows:

1. If the unsolicited message is not marked as having a duplicate message counter, has a registered Protocol ID, and the I Flag is set:

- a. Create a *new exchange* from the incoming message.
- b. The new exchange will be used by the upper layer for generating responses and subsequent processing of the message.

2. Otherwise, if the message has the R Flag set:

- a. Create an *ephemeral exchange* from the incoming message and send an immediate standalone acknowledgement.
- b. The message SHALL NOT be forwarded to the upper layer, and excluding the sending of an immediate standalone acknowledgment, SHALL be ignored.

c. The *ephemeral exchange* created for such duplicate or unknown messages with R Flag set is automatically closed in Section 4.12.5.2.2, "Standalone acknowledgement processing".

3. Otherwise, processing of the message SHALL stop.

- **Point 1** is a valid, unsolicited message in attempt to create an Exchange
- Attacker will try **Point 2**
- **Point 2.c** states that such session will automatically be closed for such **duplicate/unknown** messages in **Standalone acknowledgement processing**

Standalone acknowledgement processing

1. If the **R Flag** is set, the received message is requesting an acknowledgement be sent back:
 - a. If the message is marked as a duplicate:
 - i. Immediately send a **standalone acknowledgment**.
 - ii. If the Exchange is marked as an **ephemeral exchange** the Exchange SHALL be closed.
 - iii. Drop the message.
 - b. Otherwise, instead of sending an acknowledgement immediately upon the receipt of a reliable message from a peer, the receiver SHOULD wait for a time no longer than **MRP_STANDALONE_ACK_TIMEOUT** before sending a **standalone acknowledgment**:
 - i. Add the message counter of the received message to the **acknowledgement table** to signal that an outbound acknowledgement is pending. There can be only one outstanding acknowledgement at a time on a single **Exchange**. If a pending acknowledgement already exists for the Exchange, and it has **StandaloneAckSent** set to false, a **standalone acknowledgment** SHALL be sent immediately for that pending message counter, and the **acknowledgement table** entry SHALL be replaced for the new message.
 - ii. Start the acknowledgement timer for the Exchange.

- A. If the timer triggers before being cancelled, a **standalone acknowledgment** SHALL be sent to the source of the message. Sending this standalone acknowledgment SHALL NOT remove the **acknowledgement table** entry and SHALL set the **StandaloneAckSent** field of the entry to true.
2. The received message is then delivered to the next processing step of **Section 4.7.2, "Message Reception"**.
- **Standalone acknowledgment processing** mentions about closing the Exchange for the **duplicate** message case in **Point 1.a**
 - But no mentioning of closing the Exchange for the **unknown** message case
 - Instead, **Point 1.b** starts with "Otherwise" which implies all cases except the title of **Point 1.a**, namely "If the message is marked as **duplicate**"
 - Then **Point 1.b** mentions waiting for a period. Attackers may cause devices to waste their resources via this path

===== **Part 2** =====

4.10.5.2. Unsolicited Message Processing

An unsolicited message is processed as follows:

1. If the unsolicited message is not marked as having a duplicate message counter, has a registered Protocol ID, and the I Flag is set:
 - a. Create a *new exchange* from the incoming message.
 - b. The new exchange will be used by the upper layer for generating responses and subsequent processing of the message.
2. Otherwise, if the message has the R Flag set:
 - a. Create an *ephemeral exchange* from the incoming message and send an immediate standalone acknowledgement.
 - b. The message SHALL NOT be forwarded to the upper layer, and excluding the sending of an immediate standalone acknowledgment, SHALL be ignored.
 - c. The *ephemeral exchange* created for such duplicate or unknown messages with R Flag set is automatically closed in Section 4.12.5.2.2, “Standalone acknowledgement processing”.
3. Otherwise, processing of the message SHALL stop.

Let's focus on **Point 1** and **Point 2**. They both cover a series of steps in case the condition of respective Points are true.

```
306 // If it's not a duplicate message, search for an unsolicited message handler if it is marked as being sent by an initiator.
307 // Since we didn't find an existing exchange that matches the message, it must be an unsolicited message. However all
308 // unsolicited messages must be marked as being from an initiator.
309 if (!msgFlags.Has(MessageFlagValues::kDuplicateMessage) && payloadHeader.IsInitiator()) { ... }
310 // Discard the message if it isn't marked as being sent by an initiator and the message does not need to send
311 // an ack to the peer.
312 else if (!payloadHeader.NeedsAck()) { ... }
```

The if-block covers all three checks of **Point 1** (two checks inside if-condition parentheses, and the remaining one inside if-block)

```
306 // If it's not a duplicate message, search for an unsolicited message handler if it is marked as being sent by an initiator.
307 // Since we didn't find an existing exchange that matches the message, it must be an unsolicited message. However all
308 // unsolicited messages must be marked as being from an initiator.
309 if (!msgFlags.Has(MessageFlagValues::kDuplicateMessage) && payloadHeader.IsInitiator())
310 {
311     // Search for an unsolicited message handler that can handle the message. Prefer handlers that can explicitly
312     // handle the message type over handlers that handle all messages for a profile.
313     matchingUMH = nullptr;
314
315     for (auto & umh : UMHHandlerPool)
316     {
317         if (umh.IsInUse() && payloadHeader.HasProtocol(umh.ProtocolId))
318         {
319             if (umh.MessageType == payloadHeader.GetMessageType())
320             {
321                 matchingUMH = &umh;
322                 break;
323             }
324
325             if (umh.MessageType == kAnyMessageType)
326                 matchingUMH = &umh;
327         }
328     }
}
```

- However, the check inside “else if” does not cover **Point 2**

- In fact, it covers the opposite condition of **Point 2**
- Moreover, **Point 2** is nowhere covered

Limitation:

Currently, we do not know how to take code flow to **Exchange code blocks** in **dynamic testing**. The documentation does not possess any such section regarding this. MAILING is the only option for now.

=====

=====

=====

3 (Message Counter Sync Protocol)

P-216

The sender of the *MsgCounterSyncRsp* response message SHALL:

3. Use the same **exchange context** as the *MsgCounterSyncReq* being responded to.
 - a. The **Exchange ID** of the message SHALL be set to the *Exchange ID* of the *MsgCounterSyncReq*.
 - b. The **I Flag** SHALL be set to **0**.
 - c. The **A Flag** SHALL be set to **1**.
 - d. The **R Flag** SHALL be set to **1**.

R Flag (1 bit, position 2)

The Reliability (R) flag is a single bit field which, when set, SHALL indicate that the message sender wishes to receive an acknowledgement for the message.

- So, the sender of the **MsgCounterSyncRsp** message wishes to receive an ACK
- But the receiver of the same message does not send any ACK **as per the figure of next page**
- There is a sequence diagram in **P-218**. That also does not show sending of any ACK to the sender of the **MsgCounterSyncRsp** message

The receiver of the *MsgCounterSyncRsp* message SHALL:

1. Verify the *MsgCounterSyncRsp* matches a previously sent *MsgCounterSyncReq*:
 - a. An active synchronization exchange SHALL exist with the source node.
 - b. The *Exchange ID* field SHALL match the *Exchange ID* used for the original *MsgCounterSyncReq* message.
 - c. The *Response* field SHALL match the *Challenge* field used for the original *MsgCounterSyncReq* message.
 - d. The [Destination Node ID](#) field SHALL match the [Source Node ID](#) of the original *MsgCounterSyncReq* message.
 - e. The [Source Node ID](#) field SHALL match the [Destination Node ID](#) of the original *MsgCounterSyncReq* message.
2. On verification failure:
 - a. Silently ignore the *MsgCounterSyncRsp* message.
3. On verification success:
 - a. Set the peer's group key data message counter to *Synchronized Counter*.
 - b. Clear the [Section 4.6.5.1, "Message Reception State"](#) bitmap for the peer.
 - c. Mark the peer's group key data message counter as synchronized.

- d. Resume processing of any queued message that triggered synchronization according to [Section 4.6.7, "Counter Processing of Incoming Messages"](#).
 - i. If more than one message is queued from the synchronized peer, using the same operational group key, the messages SHALL be processed in the order received.
- e. Close the synchronization exchange created for the original *MsgCounterSyncReq* message.

```

274 CHIP_ERROR MessageCounterManager::HandleMsgCounterSyncResp(Messaging::ExchangeContext * exchangeContext,
275                                                            System::PacketBufferHandle && msgBuf)
276 {
277     CHIP_ERROR err = CHIP_NO_ERROR;
278
279     uint32_t syncCounter = 0;
280
281     const uint8_t * resp = msgBuf->Start();
282     size_t resplen      = msgBuf->DataLength();
283
284     ChipLogDetail(SecureChannel, "Received MsgCounterSyncResp response");
285
286     VerifyOrDie(exchangeContext->HasSessionHandle());
287
288     VerifyOrExit(msgBuf->DataLength() == kSyncRespMsgSize, err = CHIP_ERROR_INVALID_MESSAGE_LENGTH);
289
290     VerifyOrExit(resp != nullptr, err = CHIP_ERROR_MESSAGE_INCOMPLETE);
291     VerifyOrExit(resplen == kSyncRespMsgSize, err = CHIP_ERROR_INVALID_MESSAGE_LENGTH);
292
293     syncCounter = chip::Encoding::LittleEndian::Read32(resp);
294     VerifyOrExit(syncCounter != 0, err = CHIP_ERROR_READ_FAILED);
295
296     // Verify that the response field matches the expected Challenge field for the exchange.
297     err =
298         exchangeContext->GetSessionHandle()->AsSecureSession()->GetSessionMessageCounter().GetPeerMessageCounter().VerifyChallenge(
299             syncCounter, FixedByteSpan<kChallengeSize>(resp));
300     SuccessOrExit(err);
301
302     // Process all queued incoming messages after message counter synchronization is completed.
303     ProcessPendingMessages(exchangeContext->GetSessionHandle()->AsSecureSession()->GetPeerNodeId());
304
305     exit:
306     if (err != CHIP_NO_ERROR)
307     {
308         ChipLogError(SecureChannel, "Failed to handle MsgCounterSyncResp message with error:%" CHIP_ERROR_FORMAT, err.Format());
309     }
310
311     return err;
312 }

```

- Code also shows **no sending of any ACK** in **HandleMsgCounterSyncResp()**
- Moreover, **SendMsgCounterSyncResp()** is not even fully implemented (**figure below**)

```

230 CHIP_ERROR MessageCounterManager::SendMsgCounterSyncResp(Messaging::ExchangeContext * exchangeContext,
231                                                           FixedByteSpan<kChallengeSize> challenge)
232 {
233     System::PacketBufferHandle msgBuf;
234     VerifyOrDie(exchangeContext->HasSessionHandle());
235
236     VerifyOrReturnError(exchangeContext->GetSessionHandle()->IsGroupSession(), CHIP_ERROR_INVALID_ARGUMENT);
237
238     // NOTE: not currently implemented. When implementing, the following should be done:
239     // - allocate a new buffer: MessagePacketBuffer::New
240     // - setup payload and place the local message counter + challenge in it
241     // - exchangeContext->SendMessage(Protocols::SecureChannel::MsgType::MsgCounterSyncResp, ...)
242     //
243     // You can view the history of this file for a partial implementation that got
244     // removed due to it using non-group sessions.
245
246     return CHIP_ERROR_NOT_IMPLEMENTED;
247 }

```

Limitation:

Currently, we do not know how to take code flow to **MsgCounterSyncProtocol** code blocks in **dynamic testing**. The documentation does not possess any such section regarding this. MAILING is the only option for now.

New | A

1

P-268

5.3.5. IdentificationDeclaration Message

```
instanceName          : OCTET STRING [ length 17 ]
IdentificationDeclaration-struct => STRUCTURE [ tag-order ]
{
  VendorId             [1, optional] : UNSIGNED INTEGER [ range 16-bits ],
  ProductId            [2, optional] : UNSIGNED INTEGER [ range 16-bits ],
  DeviceName           [3, optional] : STRING [ length 0..32 ],
  DeviceType           [4, optional] : UNSIGNED INTEGER [ range 32-bits ],
  PairingInstruction    [5, optional] : STRING [ length 0..32 ],
  PairingHint           [6, optional] : UNSIGNED INTEGER [ range 32-bits ],
  RotatingDeviceId     [7, optional] : STRING [ length 0..100 ].
  Port                 [8, optional] : UNSIGNED INTEGER [ range 16-bits ],
  TargetAppList        [9, optional] : ARRAY OF
  {
    TargetApp           [10, optional] : STRUCTURE [ tag-order ]
    {
      AppVendorId       [11, optional] : UNSIGNED INTEGER [ range 16-bits ],
      AppProductId      [12, optional] : UNSIGNED INTEGER [ range 16-bits ],
```

```
    },
  },
  NoPasscode           [13, optional] : BOOLEAN,
  CdUponPasscodeDialog [14, optional] : BOOLEAN,
  CommissionerPasscode [15, optional] : BOOLEAN,
  CommissionerPasscodeReady [16, optional] : BOOLEAN,
  CancelPasscode       [17, optional] : BOOLEAN
}
```

P-269

When the Commissionee provides the **VidList**, the Commissioner SHALL apply this list as a further restriction upon the set of Content Application Vendor IDs that can be used for authentication (see AccountLogin cluster).

Current Progress of Analysis:

- No variable called **VidList** present in **IdentificationDeclaration** struct of P-268
- No such variable found in entire codebase either
- A possibility is that they referred “**AppVendorId** of **TargetApp** of **TargetAppList**” as **VidList**. HOWEVER, they did not explicitly mention of such usage anywhere

Remaining ToDo:

Investigate codebase to find which variable does the works of VidList mentioned in the spec

=====

2

P-275 (check the first red of first line)

4. When the IdentificationDeclaration message includes the **CdcForPasscode** field set to true:
 - a. Construct the **PasscodeDialogDisplayed** and set to true to indicates that a Passcode input dialog is now displayed for the user on the Commissioner. **Upon receipt, the Commissionee SHOULD display its Passcode to the user.** This field SHALL NOT be set when the CommissionerPasscode field is set because these fields are mutually exclusive.

Current Progress of Analysis:

- No variable called **CdcForPasscode** present in **IdentificationDeclaration** struct of P-268
- No such variable found in entire codebase either

Remaining ToDo:

Investigate codebase to find which variable does the works of VidList mentioned in the spec

=====

3

P-274

Table 50. List of IdentificationDeclaration parameters

Parameter Name	Feature	Description
ErrorCode	All	Indicates errors incurred during commissioning.
NeedsPasscode	Coordinate PIN Dialogs	When NoPasscode field set to true, and the Commissioner determines that a Passcode code will be needed for commissioning.
NoAppsFound	Target Content Application	No apps with AccountLogin cluster implementation were found for the last IdentificationDeclaration request. Only apps which provide access to the vendor id of the Commissionee will be considered.
PasscodeDialogDisplayed	Coordinate PIN Dialogs	A Passcode input dialog is now displayed for the user on the Commissioner.
CommissionerPasscode	Commissioner-Generated Passcode	A Passcode is now displayed for the user by the Commissioner.
QRCodeDisplayed	Commissioner-Generated Passcode	The user experience conveying a Passcode to the user also displays a QR code.
CancelPasscode	Coordinate Passcode Dialogs	Flag to indicate whether the Commissioner user has decided to exit the commissioning process.

P-275 (check the 1st and 2nd sentence of 4.a)

4. When the IdentificationDeclaration message includes the **CdcForPasscode** field set to true:
 - a. Construct the **PasscodeDialogDisplayed** and set to true to indicate that a Passcode input dialog is now displayed for the user on the Commissioner. **Upon receipt, the Commissionee SHOULD display its Passcode to the user.** This field SHALL NOT be set when the **CommissionerPasscode** field is set because these fields are mutually exclusive.

P-277

5.3.7.3.2. UDC with coordinated Passcode prompt, Passcode is needed

1. UDC with IdentificationDeclaration sent to Commissioner containing CdUponPasscodeDialog set to true
2. If Port not provided, then Commissioner reads instanceName, performs Commissionable Node Discovery to determine vid, pid, rotating ID, Device name, PASE IP, PASE Port
3. Commissioner prompts user for permission to commission Device name
4. User approves
5. Commissioner checks for apps granting access to the vid of the Commissionee, applies a filter on this list if the VidList in the IdentificationDeclaration is populated, and attempts to use Account Login on any resulting apps in order to obtain Passcode using rotatingID
6. No Passcode is obtained, Commissioner prompts user to input Passcode from Commissionee
7. Commissioner sends CommissionerDeclaration with the PasscodeDialogDisplayed set to true
8. Commissionee displays Passcode code for user
9. User enters Passcode code into Commissioner
10. Commissioner attempts to commission Commissionee

P 278, 279 (showing only relevant points)

5.3.7.5. UDC with Commissioner-generated Passcode

7. Commissioner checks for apps granting access to the vid of the client, applies a filter on this list if the VidList in the IdentificationDeclaration is populated, and attempts to use Account Login on any resulting apps in order to obtain Passcode using rotatingID
8. If no Passcode is obtained, Commissioner displays a Passcode
9. Commissioner sends CommissionerDeclaration with PasscodeDialogDisplayed and CommissionerPasscode set to true. If a QR code is also displayed, then QRCodeDisplayed is also set to true.
10. Commissionee prompts user to input Passcode from Commissioner
11. Commissionee generates a PAKE verifier using input Passcode, enters commissioning mode using this PAKE verifier
12. Commissionee sends IdentificationDeclaration to Commissioner containing CommissionerPasscode and CommissionerPasscodeReady set to true
13. Commissioner attempts to commission Commissionee using Passcode

Current Progress of Analysis:

P-274 and P-279 are consistent (**commissioner displaying the passcode**). If the table entry PasscodeDialogDisplayed suggests that passcode dialog is to be displayed in commissioner's side, then

- (1) Why does commissionee display the passcode?
- (2) How does commissionee avail the passcode?

Remaining ToDo:

Investigate further to reach a conclusion on whether this is really True Positive

=====

=====

=====

4

P-284

5.4.2.4.6. TLV Example

Extended data containing just a Rotating Device Identifier would be encoded as the following bytes:

Offset	Data	Comments
0x00	0x15	Control byte for structure with anonymous tag
0x01	0x30	Control byte for octet string with 1-byte length and a context-specific tag
0x02	0x00	Context-specific tag for Rotating Device Identifier
0x03	0x12	Length of Rotating Device Identifier (e.g. 18 bytes)
0x04	0xXX..0xXX	Rotating Device Identifier
0x16	0x18	End of container

src/setup_payload/AdditionalDataPayload.h

```
40     namespace chip {
41     namespace SetupPayloadData {
42         ...
43         inline constexpr uint8_t kRotatingDeviceIdLength = 18;
44         inline constexpr uint8_t kRotatingDeviceIdTag    = 0x00;
45         ...
46         struct AdditionalDataPayload
47         {
48         public:
49             std::string rotatingDeviceId;
50         };
```

src/setup_payload/AdditionalDataPayloadGenerator.h

```

36  namespace RotatingDeviceId {
37      static constexpr unsigned kLifetimeCounterSize = 2;
38      static constexpr unsigned kHashSuffixLength    = 16;
39      static constexpr unsigned kMaxLength           = kLifetimeCounterSize + kHashSuffixLength;
40      static constexpr unsigned kHexMaxLength        = kMaxLength * 2 + 1;
41      } // namespace RotatingDeviceId
42      :
43  enum class AdditionalDataFields : int8_t
44  {
45      NotSpecified      = 0x00,
46      RotatingDeviceId  = 0x01
47  };

```

Current Progress of Analysis:

- What is 0xXX..0xXX? It seems like two hex values are merged
- What is the range of **RotatingDeviceId**? Spec does not mention anything. It only mentions this SHALL be of 128 bits
- Code does not help with indicating the size or value of **RotatingDeviceId**

Remaining ToDo:

Investigate further to reach a conclusion on whether this is really True Positive

AII B

1

P-69

2.11.2.2. Subscribe Interaction Limits

- A publisher SHALL ensure that every fabric the node is commissioned into can support **at least three Subscribe Interactions** to the publisher and that each subscription SHALL support **at least 3 attribute/event paths**.
- A server MAY permit **Subscribe Interactions** even when there is no accessing fabric, subject to available resources (e.g over PASE).
- Device type specifications MAY require a larger number of supported subscriptions or paths.
- **SUBSCRIPTION_MAX_INTERVAL_PUBLISHER_LIMIT** defines the upper limit for the publisher-selected maximum interval for any subscription.
 - If the publisher is an **ICD**, this SHALL be set to the **Idle Mode Duration**.
 - If not, this SHALL be set to 60 minutes.
- **The minimal supported capabilities, subject to the minimal constraints above, are reported in the CapabilityMinima attribute of the Basic Information cluster.**

The blue-marked parts are the only minimal constraints in this **subsection 2.11.2.2** which are referred to in the yellow-marked part. Now, we need to cross-match this info with the **Basic Information cluster** as this cluster has been referred to in the yellow-marked part.

P-732

11.1.4.4. CapabilityMinimaStruct Type

This structure provides constant values related to overall global capabilities of this Node, that are not cluster-specific.

Matter Specification R1.4

Connectivity Standards Alliance Document 23-27349 November 4,

ID	Name	Type	Constraint	Quality	Default	Access	Conformance
0	CaseSessionsPerFabric	uint16	min 3		3		M
1	SubscriptionsPerFabric	uint16	min 3		3		M

CaseSessionsPerFabric Field

This field SHALL indicate the actual minimum number of concurrent [CASE](#) sessions that are supported per fabric.

This value SHALL NOT be smaller than the required minimum indicated in [Section 4.14.2.8](#), “Minimal Number of CASE Sessions”.

SubscriptionsPerFabric Field

This field SHALL indicate the actual minimum number of concurrent [subscriptions](#) supported per fabric.

This value SHALL NOT be smaller than the required minimum indicated in [Section 8.5.1](#), “Subscribe Transaction”.

As we can see, the variable **SubscriptionsPerFabric** matches with the aforementioned **subsection 2.11.2.2**, but the variable **CaseSessionsPerFabric** is completely missing in **subsection 2.11.2.2**.

Likewise, “each subscription SHALL support at least 3 attribute/event paths” constraint is not covered in this **Basic Information cluster**.

Code Status: It complies with P-732

Next Step: Unknown

###2

P 176, 177

To make the resumption succeed, both the Initiator and the Responder SHALL have remembered the SharedSecret they have computed during the previous execution of the CASE session establishment. It SHALL be that SharedSecret that is used to compute the resumption ID.

Session Resumption State

To perform session resumption, the following state from the [previous session context](#) must be known to the initiator and responder:

1. SharedSecret
2. Local Fabric Index
3. Peer Node ID
4. Peer CASE Authenticated Tags
5. ResumptionID

Current Progress of Analysis:

If ResumptionID can be computed by SharedSecret, why is it necessary that ResumptionID is one of the five states that must be known for session resumption?

Remaining ToDo:

Check codebase

###3

P-308

5.7.3.1.1. Expansion of CommissioningCustomFlowUrl by Commissioner

Once the URL is obtained, it SHALL be expanded to form a final URL (**ExpandedCommissioningCustomFlowUrl**) by proceeding with the following substitution algorithm on the original **CommissioningCustomFlowUrl**:

1. If key **MTcu** is present, compute the **CallbackUrl** desired (see [Section 5.7.3.2, “CallbackUrl format for Custom Commissioning Flow response”](#)), and substitute the placeholder value “_” (i.e. in **MTcu=_**) in the **CommissioningCustomFlowUrl** with the desired contents, encoded with proper URL-encoded escaping (see [RFC 3986 section 2](#)).
2. If key **MTop** is present, substitute the placeholder value “_” (i.e. in **MTop=_**) in the **CommissioningCustomFlowUrl** with either **numeric manual code**, or **QR code body** including the **MT**: prefix and TLV data (if present), encoded with proper URL-encoded escaping (see [RFC 3986 section 2](#)). Note that both methods SHOULD be supported by the Manufacturer’s custom flow.

A Commissioner SHALL NOT append the **MTop=** query key/value pair unless the key/value pair was already present as **MTop=_** in the **CommissioningCustomFlowUrl** previously obtained. This constraint enables the determination of which products make use of the payload in their Custom Commissioning Flow infrastructure by inspection of the Distributed Compliance Ledger records.

The final URL after expansion (**ExpandedCommissioningCustomFlowUrl**) SHALL be the one to follow per [Section 5.7.3, “Custom Commissioning Flow”](#), rather than the original value obtained from the Distributed Compliance Ledger.

Current Progress of Analysis:

There should have been a sentence about **MTcu** as well, similar to the blue-marked one (i.e. **“A Commissioner SHALL NOT append the MTcu= query key/value pair unless the key/value pair was already present as MTcu=_ in the CommissioningCustomFlowUrl previously obtained”** should have been there as well)

Remaining ToDo:

- Go through the spec to confirm if this sentence really should have been in the spec
- Check codebase

New v2

Remarks on Inconsistency:

According to P-622, the action's state SHALL change to either Inactive or Active, depending on the actions (see examples 4 and 6). **However, in both examples — 4 & 6, the state changed to only Inactive state.** Moreover, there was no check that represents the “depending on the actions” part.

1. The Specification Inconsistency

- **The Normative Text:** Section 9.14.6.12 states that once the **Duration** for a **DisableActionWithDuration** command expires, the action **SHALL** change to either **Inactive** or **Active**, depending on the action's internal logic.
- **The Visual Diagrams:** Example 4 (Wake-up) and Example 6 (Alarm) only show a transition arrow (edge) pointing from **Disabled -> Inactive**. They do not visually represent a direct jump to **Active**.

2. The Code Implementation Findings

- **Pattern:** The code in **actions-server.cpp** utilizes a **Delegation Pattern**. The server does not force a state; it waits for the **Delegate** (the device application) to report its new status.
- **The OnStateChanged Event:** The function **OnStateChanged(..., ActionStateEnum aActionState)** is designed to accept **any** valid state from the enum. It does not contain a "gatekeeper" or logic check that forces a Disabled action to become Inactive first.
- **Execution Flow:** When the timer (Duration) expires within the application layer, the device evaluates its current requirements. If the action is "due" to be running, the application sets the state to **Active** and notifies the server. The server then generates the **StateChanged** event for the network.

3. Final Conclusion

The **Text** in the specification is the correct functional requirement, while the **Diagrams** are simplified examples of a "reset to idle" flow. The **Source Code** is written to support the **Text**, providing a flexible reporting mechanism that allows a direct **Disabled -> Active** transition without an intermediate Inactive step.

=====
=====
=====

2

P-667

9.17.6. Attributes

ID	Name	Type	Constraint	Quality	Default	Access	Conformance
0x0000	IdleMode-Duration	uint32	1 to 64800	F	1	R V	M
0x0001	Active-ModeDuration	uint32	all	F	300	R V	M
0x0002	Active-Mode-Threshold	uint16	all	F	300	R V	M

P-656

9.16.1.6.5. Runtime Polling Interval Changes

In addition to being able to dynamically switch between LIT and SIT operating modes, an ICD can also dynamically change its Fast Polling Interval, Slow Polling Interval and its Active Mode Threshold. A change in Active Mode Threshold value SHALL trigger a DNS-SD update of the SAT TXT key. Changes of the SESSION_ACTIVE_INTERVAL or SESSION_IDLE_INTERVAL SHALL also trigger a DNS-SD update of the corresponding SAI or SII TXT keys. Note that an update of the polling intervals MAY NOT trigger an update of the SAI or SII TXT keys if the new polling intervals are still compliant with the rules from ICD Session Configurations section.

Remarks on Inconsistency:

According to the blue-marked part of P-656, **ActiveModeThreshold** can be changed. But as per table of P-667, the **Quality** of this attribute is **F (Fixed)**.

While the attribute value stored in the cluster is "Fixed" (read-only and constant for that firmware version), the actual value used in the network (advertised via DNS-SD) can change dynamically to reflect the device's current state. It is not a "timer" that counts down, but rather a configuration value that defines the length of time the device will stay awake after a message exchange.

WIP

=====
=====
=====

3

P-895

11.18.4.4. NOCStruct Type

This encodes a [fabric sensitive NOC](#) chain, underpinning a commissioned Operational Identity for a given Node.

Access Quality: Fabric Scoped							
ID	Name	Type	Constraint	Quality	Default	Access	Conformance
1	NOC	octstr	max 400			S	M
2	ICAC	octstr	max 400	X		S	M

Note that the [Trusted Root CA Certificate](#) is not included in this structure. The roots are available in the [TrustedRootCertificates](#) attribute of the Node Operational Credentials cluster.

NOC Field

This field SHALL contain the [NOC](#) for the struct's [associated fabric](#), encoded using [Matter Certificate Encoding](#).

ICAC Field

This field SHALL contain the [ICAC](#) or the struct's [associated fabric](#), encoded using [Matter Certificate Encoding](#). If no ICAC is present in the chain, this field SHALL be set to null.

Remarks on Inconsistency:

In congruence with the blue one, the red one should be —

This field SHALL contain the ICAC **“for”** the struct's associated fabric (NOT This field SHALL contain the ICAC **“or”** the struct's associated fabric)

=====
=====
=====

4

P-945

DownloadProtocolEnum Type

This data type is derived from [enum8](#).

Value	Name	Summary	Conformance
0	BDXSynchronous	Indicates support for synchronous BDX.	M
1	BDXAsynchronous	Indicates support for asynchronous BDX.	O

Value	Name	Summary	Conformance
2	HTTPS	Indicates support for HTTPS.	O
3	VendorSpecific	Indicates support for vendor specific protocol.	O

P-947

ProtocolsSupported Field

This field SHALL contain a list of all download protocols supported by the OTA Requestor.

This field SHALL be used by the OTA Provider to generate the correct URI for the location of the Software Image when one is found to be available. **The values of BDX Synchronous and BDX Asynchronous SHALL always be supported by an OTA Provider.** Furthermore, OTA Providers with access to external networking SHOULD support the HTTPS protocol. OTA Providers MAY support other protocols.

Remarks on Inconsistency:

According to the yellow-marked part of P-947, **BDXAsynchronous** is a mandatory feature. However, as per P-945, its **Conformance** is **O (Optional)**.

In the codebase after searching for these attributes we could only find **BDXSynchronous** in `/src/app/clusters/ota-requestor/DefaultOTARequestor.cpp` and `/src/controller/python/matter/clusters/Objects.py` but still no conclusion can be drawn.

=====

=====

=====

5

P-980

Field	Size	Notes
Message ID: ReceiveAccept(0x05)		
Transfer Control (TC)	1 octet	Must specify exactly one of: • TC[ASYNC] • TC[RECEIVER_DRIVE] • TC[SENDER_DRIVE]
Range Control (RC)	1 octet	
Max Block Size (MBS)	2 octets	Unsigned little-endian. Must be $\leq$ PMBS .
Length (LEN)	4/8 octets	Optional. • Present if RC[DEFLen] = 1 • Size = 4 if RC[WIDERANGE] = 0 • Size = 8 if RC[WIDERANGE] = 1
Metadata (MDATA)	variable	Optional, TLV.

The following subsections describe the fields composing the **SendAccept** and **ReceiveAccept** messages.

Transfer control (TC)

The Transfer Control (TC) field specifies, as subfields, the transfer mode and protocol version.

For the transfer mode (**TC[SENDER_DRIVE]**, **TC[RECEIVER_DRIVE]**), exactly one mode SHALL be chosen for this transfer, which MUST be one of the original proposed transfer methods sent by the Initiator.

Remarks on Inconsistency:

According to the blue-marked part, there are three modes — **TC[ASYNC]**, **TC[SENDER_DRIVE]**, **TC[RECEIVER_DRIVE]**. Why is **TC[ASYNC]** not mentioned in the red-marked part.

In the codebase src/controller/python/matter/bdx/[BdxProtocol.py](#) all of the three modes can be found. But in the test cases src/app/tests/suites/certification/*.yaml they only mentioned RECEIVER_DRIVE and SENDER_DRIVE.

6

P-1030

FriendlyName Field

Friendly name for this endpoint which is propagated to nodes. Any changes to Friendly Name or Group Id List (add/remove entry) must follow the pending→committed workflow with current state reflected in the Status Entry.

StatusEntry Field

Indicates whether changes to Friendly Name are pending or committed.

GroupIDList Field

List of Group IDs that this endpoint is a member of. Any changes to Group Id List (add/remove entry) must follow the pending→committed workflow with current state reflected in the Status Entry.

Remarks on Inconsistency:

- According to the description of **StatusEntry** Field, it is related to changes of **FriendlyName** only
- However, as per the description of **GroupIDList** Field, it handles changes of **GroupIDList** too
- **Comment:** Everything would be okay, if the 2nd sentence of the description of **FriendlyName** Field (*“Any changes to Friendly Name or Group Id List (add/remove entry) must follow the pending→committed workflow with current state reflected in the StatusEntry.”*) were in the definition of **StatusEntry** Field.

1. The Code Implementation Findings

- **Architectural Choice:** The code in [JointFabricDatastore.cpp](#) uses **Isolation** instead of **Aggregation**.

- **Top-Level Status:** The main `commissioningStatusEntry.state` is used almost exclusively as a "Node Busy" flag during the `RefreshNode` process. When `UpdateNode` changes a `FriendlyName`, the code does not automatically set the main status back to `kPending`.
- **Lower-Level Statuses:** The `GroupIDList` and `BindingList` entries are structures that contain **their own internal `statusEntry`**.
 - Functions like `AddGroupIDToEndpointForNode` update the **internal** status of that specific group to `kPending`.
 - Crucially, these functions **do not** call `SetNode()` to update the top-level `StatusEntry`.

2. Final Conclusion

The **Source Code** follows a "Siloed Status" pattern. While the **Specification** suggests the top-level `StatusEntry` should act as a "Master Summary" for all changes on that endpoint, the **Implementation** treats it as a single-purpose flag for the Node/Name.

This results in a logical gap: a controller checking only the top-level `StatusEntry` may see a **Committed (Green)** state, even if Group or Binding changes are still **Pending (Yellow)** within the sub-lists. The implementation effectively prioritizes the "Field Definition Rule" over the "Dependency Rule."

Moreover, jointfabric feature is not fully complete and there are many todo things in the code.

DEAD

1

4.3.1.2. Extended Discovery

A Matter Commissionee that advertises Commissionable Node Discovery service records is not necessarily in a state that will allow Commissioning (this state is referred to below as "in Commissioning Mode"). While Section 5.4.2.3, "Announcement Duration" is limited for some forms of device advertisement, a Matter device MAY advertise Matter Commissionable Node Discovery service records for longer periods, possibly permanently. Advertising Commissionable Node Discovery when not in Commissioning Mode is referred to here as Extended Discovery.

Extended Discovery is allowed only for DNS-SD advertisements and not for the other forms of Device Discovery such as BLE Commissioning Discovery.

To protect customer privacy on public networks, a Matter Commissionee SHALL provide a way for the customer to set a timeout on Extended Discovery, or otherwise disable Extended Discovery. The default behavior for Commissionable Node Discovery SHOULD default to limiting announcement as defined in Section 5.4.2.3, "Announcement Duration" unless the Manufacturer wishes to enable longer periods for specific use cases.

5.4.2.5.3. Interval

Commissionable devices SHOULD use an Advertising Interval between 20 ms and 60 ms for the first 30 seconds and a value between 150 ms to 1285 ms for the rest of the Announcement duration. Shorter intervals typically result in shorter discovery times.

If a device opts to use Extended Announcement, it SHALL switch to using an Advertising Interval larger or equal to 1200 ms and SHOULD use a nominal Advertising Interval of 1285 ms. When using Extended Announcement, the device SHALL set the Extended Announcement Flag in the Matter Service Data in the BLE Advertisement (see Table 51, "Matter BLE Service Data payload format").

Verdict: CONTRADICTION

Relation: 2

Reason: Text-1 states that "Extended Discovery" is allowed only for DNS-SD advertisements and not for BLE commissioning discovery, whereas Text-2 specifies that when a device opts for "Extended Announcement" (which applies to BLE advertisements) it must use a large advertising interval and set a flag. The two sections therefore give conflicting guidance on whether extended discovery/announcement can be used with BLE, leading to a contradiction.

Evidence:

esp32/KConfig

```
577     config CHIP_BLE_EXT_ADVERTISING
578         bool "Bluetooth LE extended advertising"
579         help
580             Enable Bluetooth LE extended advertising, which allows the device to advertise
581             Matter service over Bluetooth LE for a period of time longer than 15 minutes.
582             If this config is true,
583             CHIP_DEVICE_CONFIG_DISCOVERY_TIMEOUT_SECS define can be set up to 172800 seconds (48h).
```

nxp/cmake/KConfig.matter.common

```
511     config CHIP_BLE_EXT_ADVERTISING
512         bool "Bluetooth LE extended advertising"
513         help
514             Enable Bluetooth LE extended advertising, which allows the device to advertise
515             Matter service over Bluetooth LE for a period of time longer than 15 minutes.
516             If this config is true,
517             CHIP_DEVICE_CONFIG_DISCOVERY_TIMEOUT_SECS define can be set up to 172800 seconds (48h).
```

zephyr/KConfig

```
577     config CHIP_BLE_EXT_ADVERTISING
578         bool "Bluetooth LE extended advertising"
579         help
580             Enable Bluetooth LE extended advertising, which allows the device to advertise
581             Matter service over Bluetooth LE for a period of time longer than 15 minutes.
582             If this config is true,
583             CHIP_DEVICE_CONFIG_DISCOVERY_TIMEOUT_SECS define can be set up to 172800 seconds (48h).
```

efr32_sdk.gni

```
57         # Enable the BLE extended advertisement
58         sl_matter_ble_extended_adv = false
```

PICS.yaml

```
184         - label:
185             "Does the commissionee device support Extended Discovery through
186             DNS-SD advertisements when device is not in commissioning mode?"
187             id: MCORE.DD.EXTENDED_DISCOVERY

299         - label:
300             "Does device support Extended Discovery for Commissionable Node
301             Discovery?"
302             id: MCORE.SC.EXTENDED_DISCOVERY
```

These IDs are used in Test_TC_SC_4_1.yaml

ConnectivityManager.h

```
151     enum BLEAdvertisingMode
152     {
153         kFastAdvertising      = 0,
154         kSlowAdvertising      = 1,
155         kExtendedAdvertising  = 2,
156     };
```

But BLEAdvertisingMode.kExtendedAdvertising is not used in any active code block

include/platform/ChipDeviceConfig.h

```
750 > #if CHIP_DEVICE_CONFIG_EXT_ADVERTISING Inactive Preprocessor Block
789 #endif

750 #if CHIP_DEVICE_CONFIG_EXT_ADVERTISING
751
752 /**
753  * CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_CHANGE_TIME_MS
754  *
755  * The amount of time in milliseconds after which BLE advertisement should be switched from the slow
756  * advertising to the extended advertising, counting from the moment of advertisement commencement.
757  *
758  * Defaults to 900000 ms.
759  */
760 #ifndef CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_CHANGE_TIME_MS
761 #define CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_CHANGE_TIME_MS (15 * 60 * 1000)
762 #endif
763
764 /**
765  * CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_MIN
766  *
767  * The minimum interval (in units of 0.625ms) at which the device will send BLE advertisements while
768  * in the extended advertising mode. The minimum interval shall not be smaller than the default value.
769  *
770  * Defaults to 2056 (1285 ms).
771  */
772 #define CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_MIN 2056
773
774 /**
775  * CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_MAX
776  *
777  * The maximum interval (in units of 0.625ms) at which the device will send BLE advertisements while
778  * in the extended advertising mode. The maximum interval should be greater.
779  *
780  * Defaults to 2056 (1285 ms).
781  */
782 #ifndef CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_MAX
783 #define CHIP_DEVICE_CONFIG_BLE_EXT_ADVERTISING_INTERVAL_MAX 2056
784 #endif
```

src/platform/ESP32/ChipDeviceConfig.h

```
92 #ifdef CONFIG_ENABLE_EXTENDED_DISCOVERY
93 | #define CHIP_DEVICE_CONFIG_ENABLE_EXTENDED_DISCOVERY CONFIG_ENABLE_EXTENDED_DISCOVERY
94 #else
95 | #define CHIP_DEVICE_CONFIG_ENABLE_EXTENDED_DISCOVERY 0
96 #endif // CONFIG_ENABLE_EXTENDED_DISCOVERY
```

All code blocks that include BLE Extended Discovery is inactive (due to
CHIP_DEVICE_CONFIG_ENABLE_EXTENDED_DISCOVERY = 0)

Useful:

Search with “CHIP_BLE_EXT_ADVERTISING”

<https://gemini.google.com/share/777e98704070>

#####

2

2.5.5.6. Unspecified Node ID

The Unspecified Node ID (0x0000_0000_0000_0000) is a reserved value that never appears in messages or protocol usage. It exists to mark or detect the presence of uninitialized, missing, or invalid Node IDs.

.

4.8.1.1. Nonce

The nonce used for the Authenticated Encryption with Additional Data (AEAD) algorithm (see Section 3.6, “Data Confidentiality and Integrity”) for a given message SHALL be defined as the concatenation of the Security Flags, the Message Counter, and the Source Node ID of that message. The scalar fields in the nonce, namely the Message Counter and the Source Node ID SHALL be encoded in little-endian byte order for the purposes of serialization within the nonce, that is, in the same byte ordering as the segment of the message from which its data originates.

The Source Node ID field used in the nonce SHALL be set to the Operational Node ID of the node originating security protection of the message:

- If the message is of Secure Unicast Session Type:
 - For a CASE session, the Nonce Source Node ID SHALL be determined via the Secure Session Context associated with the Session Identifier.
 - For a PASE session, the Nonce Source Node ID SHALL be Unspecified Node ID.
- If the message is of Group Session Type:
 - The S Flag of the message SHALL be 1 and the Nonce Source Node ID SHALL be the Source Node ID of the message.
 - If the S Flag of the message is 0 the message SHALL be dropped.

NOTE: Because PASE negotiates strong one-time keys per session and the I2RKey and R2IKey are distinct for each direction of communication, the use of the Unspecified Node ID as the Nonce Source Node ID remains semantically secure.

.

Verdict: CONTRADICTION

Relation: 2

Reason: Text-1 asserts that the Unspecified Node ID (0x0000...) is a reserved value that never appears in any protocol messages, whereas Text-2 explicitly states that this same value is to be used as the Source Node ID in the nonce for PASE sessions. The two statements directly conflict, making the pair contradictory.

Evidence:

PASession.h

```
68     ScopedNodeId GetLocalScopedNodeId() const override
69     {
70         // For PASE, source is always the undefined node ID
71         return ScopedNodeId();
72     }
```

ScopeNodeId.h

```
26     class ScopedNodeId
27     {
28     public:
29         ScopedNodeId() : mNodeId(kUndefinedNodeId), mFabricIndex(kUndefinedFabricIndex) {}
```

NodeId.h

```
31     inline constexpr NodeId kUndefinedNodeId = 0ULL;
```

TODO:

Search with "GetLocalScopedNodeId" in the entire workspace

kUndefinedNodeId is used in GroupSession.h, but not mentioned in the spec. Read 4.16, 4.17

#####

3

The Global Group Encrypted Message Counters are used to generate the counter for messages encrypted using group keys. There are two such counters:

- The Global Group Encrypted Data Message Counter is used to encode regular data messages encrypted with a group key.
- The Global Group Encrypted Control Message Counter is used to encode control messages encrypted with a group key.

Some Nodes might not be required to implement communication using group keys, in which case they MAY omit the Global Group Encrypted Message Counters. In contrast to the Global Unencrypted Message Counter, Nodes are required to persist the Global Group Encrypted Message Counters in durable storage. In particular, Nodes are required to ensure that the value of the Global Group Encrypted Message Counters never rolls back and that it is monotonic within the bounds of its range

4.6.1. Message Counter Types

All Nodes implement three global 32-bit counters to generate message counters for certain types of messages:

- Global Unencrypted Message Counter
- Global Group Encrypted Data Message Counter

- Global Group Encrypted Control Message Counter

Additionally, Nodes implement a separate 32-bit counter for each session as part of secure session state:

- Secure Session Message Counter

Additionally, Nodes implement a separate 32-bit counter for each Check-In Protocol use case:

- Check-In Counter

Technical details for how each counter type works are described in the following sections. Table 16, “Message Counter Type Overview” is provided to summarize higher-level differences between Message Counter Types:

Verdict: CONTRADICTORY

Relation: 2

Reason: Text-1 specifies that nodes not required to use group keys MAY omit the Global Group Encrypted counters, implying the counters are optional for some nodes. Text-2 states that all nodes implement the three global 32-bit counters, including the encrypted ones. This direct conflict between “optional” and “required” makes the two descriptions contradictory.

#####

4

6.6.2.1.2.1. Example CASE Authenticated Tags

- Joint Fabric Administrator Tag identifier 0xFFFF, Version 0x0001
 - CAT value: 0xFFFF_0001
 - Appears in Access Control Entry Subjects list as 0xFFFF_FFFD_FFFF_0001
 - Appears in Node Operational Certificate subject under OID 1.3.6.1.4.1.37244.1.6 with value 0xFFFF0001
 - Would appear in X.509 certificate subject under OID 1.3.6.1.4.1.37244.1.6 as UTF8String FFFF0001 for signature validation purposes.
- Tag identifier 0xAB12, Version 0x0003
 - CAT value: 0xAB12_0003
 - Appears in Access Control Entry Subjects list as 0xFFFF_FFFD_AB12_0003
 - Appears in Node Operational Certificate subject under OID 1.3.6.1.4.1.37244.1.6 with value 0xAB120003
 - Would appear in X.509 certificate subject under OID 1.3.6.1.4.1.37244.1.6 as UTF8String AB120003 for signature validation purposes.
- Tag identifier 0x071C, Version 0x1074
 - CAT value: 0x071C_1074
 - Appears in Access Control Entry Subjects list as 0xFFFF_FFFD_071C_1074
 - Appears in Node Operational Certificate subject under OID 1.3.6.1.4.1.37244.1.6 with value 0x071C1074
 - Would appear in X.509 certificate subject under OID 1.3.6.1.4.1.37244.1.6 as UTF8String

071C1074 for signature validation purposes.

.

6.5.1.1. ASN.1 Object Identifiers (OID)

Several important components of X.509 certificates follow the pattern commonly used in ASN.1 data models where some types are constructed with an ASN.1 object identifier (OID) to identify each variant. For example, the cryptographic algorithm used in the digital signature is identified by its OID.

Matter certificates do not use ASN.1 OIDs. Instead, each valid ASN.1 OID SHALL be mapped to a Matter TLV tag within its reference category. Each reference category defines the context of the Matter tag, and tag values are assigned to the reference categories according to the type of fields where they can appear in X.509 certificates.

.

Verdict: CONTRADICTION

Relation: 2

Reason: Text-1 explicitly states that tag values appear under the ASN.1 OID

1.3.6.1.4.1.37244.1.6 in the Node Operational Certificate subject, demonstrating that ASN.1 OIDs are employed. Text-2, however, declares that Matter certificates do not use ASN.1 OIDs at all. The two statements directly conflict, indicating an inconsistency between the texts.

#####

5

13.4. Factory Reset

- a. Devices and Nodes SHALL have a factory reset capability. [CM15 for T16, T17, T79, T82]
- b. Factory reset SHALL remove from the Node all security- and privacy-related data and key material created during or after commissioning except data explicitly required to persist across resets. [CM35 for T16, T17, T79, T82]

See the following sections for detailed factory reset requirements.

- Section 6.4.3, "Node Operational Identifier Composition"
- Section 6.4.11, "Security Considerations"
- Section 6.6.3, "Access Control List Examples"
- Section 7.12.1, "Persistence"
- Section 7.14, "Event"
- Section 11.12, "General Diagnostics Cluster"
- Section 11.20.4.2, "Image Verification"
- NOCs
- Fabrics

- CommissionedFabrics
- TrustedRootCertificates

.

7.12.1. Persistence

Persistent data retains its value across a restart.

A restart is:

- a program restart or reboot
- power cycle reboot
- user-initiated reboot
- reboot initiated from a program image upgrade

A factory reset is not such a restart. A factory reset is a deliberate behavior to reset persistent data back to its original state when the product left the factory.

Cluster attributes that represent configuration data SHALL be persistent data unless otherwise specified.

For example: a writable attribute that persistently changes the behavior (or mode) of the cluster. Examples of non-configuration data: device state data, data that is calculated or comes from an external source, such as a sensor value, a time value, etc.

Many clusters define persistent data that is not surfaced as attributes, but is managed by commands.

Commissioning or configuration data that is created to allow the cluster to perform its function is persistent data. A group table entry and binding entries are both persistent data across a restart.

When a persistent cluster attribute represents controlled state of the device, the device SHALL restore the attribute value to the value before the restart was initiated, and put the device in the state that is represented by the restored value.

For example: After an OTA cluster restart, clusters that have visible state attributes, such as the state of a light, or a window shade SHALL be persistent and define these attributes as persistent.

Some cluster specifications add a dependency with a persistent configuration attribute A that contains

a value to use to restore persistent state attribute B after a restart. This is perfectly valid but cluster specific.

Cluster state data that is not controlled, such as sensor data, is not considered persistent.

The cluster specification may put dependencies and limitations on persistent data.

.

Verdict: CONTRADICTION

Relation: 2

Reason: Text-1 says a factory reset removes all security-related data except data that “must persist across resets,” implying that such data is preserved by a factory reset. Text-2 states that a factory reset resets persistent data back to its original state, i.e., it does not preserve that data. The two descriptions conflict on whether persistent data remains after a factory reset, so the texts are contradictory.

#####

DUMP

1

P-1023

(Section 11.24.4.8): States that "Removing a Group" is performed by using the `UpdateGroup` command. It explicitly mandates that this command **"SHALL fail if the Group is in use... by any Nodes"**.

P-1038

(Section 11.24.7.5): Defines the formal functional logic for the `UpdateGroup` command. This section **completely omits** the check for whether a group is "in use". Instead, it describes a process where the Datastore iterates through all Node entries and applies changes (such as GroupKeySetID updates or ACL changes) to those Nodes

Current progress of analysis:

- There is a functional discrepancy between the "Usage Requirements" (which forbid removing a group in use) and the "Command Implementation" (which specifies how to update nodes that are using the group but does not enforce the failure condition)
- in the code implementation this check was not introduced.

Code Reference:

`./src/app/server/JointFabricDatastore.cpp::UpdateGroup()`

Remaining To-Do

- Verify which command is actually being used. There is a `RemoveGroup` command in addition to the `UpdateGroup` method, and neither appears to include the required check.
- This is intended to demonstrate that, when deleting a group, the `RemoveGroup` command was meant to be used, but instead the documentation or guidance suggests using the `UpdateGroup` command.
- Regardless of which method is used, our goal is to prove that a group can still be removed even if it is currently being used by a node.

Testing Plan

- Create a Joint Fabric Datastore by following the steps outlined here: https://github.com/project-chip/connectedhomeip/blob/master/docs/guides/joint_fabric_guide.md
- After the datastore is created, set up a lighting app and pair it with the joint-fabric controller.
- Use the relevant Joint Fabric cluster command to create a group and add the lighting app node to that group.

- Current issue: Executing this command results in an “unsupported cluster” error.

README

On 20/1/2026,

Any page reference of this doc is to Matter spec v1.4 (pdf:

https://drive.google.com/file/d/14pvZAel1REVc9dQiu3q2LboAYpelflFJ/view?usp=drive_link)

Tabs

Old | A: Inconsistencies found in the past and important ones till now

New | A: Inconsistencies found after the meeting of 15/1/2026 (period: 15/1/2026 - 19/1/2026) and important ones till now

All B: Minor inconsistencies found so far (including the ones found after the last meeting).

It will most likely not be possible to exploit these inconsistencies. However, we are yet to confirm their usefulness

New v2: Inconsistencies found in period 19/1/2026 ~ 5/2/2026

DEAD: Subset of the pairs which were thought as inconsistent once upon in a time, but proven as False Positive later on. This tab CAN be ignored

DUMP: Unimportant stuff. This tab CAN be ignored